

Minimalist Texas Hold'em · Project Structure & Implementation Guide

1 Directory Structure

```
CS181_project/
... main.py                # Entry point (3 run modes:
interactive, evaluate, step)
... requirements.txt        # Python dependencies
... cfr_policy.pkl          # Pre-trained CFR strategy cache
... README.md              # Project paper / documentation
... struct.md              # This file: structure &
implementation guide
.
... game/                  # ===== Core Game Module =====
.   ... __init__.py
.   ... constants.py       # Game constants (deck, blinds, betting
levels, action codes)
.   ... card.py            # Deck class + treys card utility
functions
.   ... evaluator.py       # Hand evaluation, comparison, equity
computation
.   ... engine.py          # Game engine: deal · betting rounds ·
showdown
.   ... cfr_solver.py      # Custom CFR solver (External Sampling
MCCFR)
.
... agents/                # ===== Agent Implementations =====
.   ... __init__.py
.   ... base_agent.py      # Abstract base class (unified
interface)
.   ... random_agent.py    # Random strategy Agent (testing
baseline)
.   ... expert_agent.py    # Expert Agent (custom CFR Nash
equilibrium strategy)
.   ... sarsa_agent.py     # SARSA online TD learning Agent
.   ... bayesian_mc_agent.py # Bayesian inference + MC Q-table
Agent
.   ... nn_mc_agent.py     # BNN (MC Dropout) + MC Q-table Agent
```

2 Game Rules

2.1 Basic Setup

Parameter	Value	Description
		----- -----
		Game mode 2-player AI vs AI Heads-up Limit Texas Hold'em
Deck	16 cards	2 suits (·) × 8 ranks (7,8,9,T,J,Q,K,A)
		Starting chips 1000 Per player
	Blinds	SB=5, BB=10
		Heads-up: dealer = small blind
		Betting levels {10, 20, 40, 80}
	Corresponding to B_level 0~3	Max raises 3 per round Highest bet = 80

2.2 Action Space

$\mathcal{A} = \{0: \text{Fold}, 1: \text{Call}, 2: \text{Raise}\}$

- **Fold (0):** Surrender the hand; opponent wins the pot
- **Call (1):** Match the current bet level; if no one has bet yet (post-flop first action), Call acts as Check
- **Raise (2):** Increase betting level by one and pay the difference; illegal when max level or 3-raise limit is reached

2.3 Game Flow

1. Post blinds
 - Dealer (dealer_pos) posts small blind 5
 - Non-dealer posts big blind 10
2. Preflop
 - Deal 2 hole cards to each player
 - Dealer (small blind) acts first
 - Betting level starts at 0 (amount = 10)
3. Flop
 - Deal 3 community cards
 - Non-dealer acts first
 - Betting level resets (first bet starts at level 0)
4. Turn
 - Deal 1 community card
 - Same action order as Flop
5. River
 - Deal 1 community card (total 5)
 - Same action order as Flop
6. Showdown
 - Best 5-card combination from 2 hole + 5 community cards
 - Evaluated by treys library (lower rank = stronger)
 - Winner takes the pot; loser loses their total bet

2.4 Hand Rankings

Possible hand types in the 16-card deck (strongest to weakest):

Straight Flush > Four of a Kind > Full House > Flush > Straight > High Card

Note: One Pair and Three of a Kind are still possible with 16 cards (2 suits $\times$ 8 ranks). treys evaluates them correctly.

2.5 Heads-up Special Rules

- Dealer = Small Blind, acts first preflop
- Non-dealer = Big Blind, acts first post-flop
- Dealer position alternates after each hand

3 MDP Formulation

3.1 State Space

$S = (H_{\text{code}}, P_{\text{code}}, B_{\text{level}}, \text{Pos})$

Component	Meaning	Encoding	-----	-----
-----	-----	H_code	Own hand equity	Discretized into 20 bins (equity_to_bin) P_code
Community info	Community card count / treys rank code	B_level	Betting level	{0, 1, 2, 3} · {10, 20, 40, 80}
Pos	Seat position	{0, 1}		

3.2 Reward Function

Sparse reward, given only at hand termination:

$R = \begin{cases} P_{\text{pot}} \cdot \text{win} - B_{\text{own}} \cdot \text{lose} & \text{if } \text{win} \neq \text{lose} \\ 0 & \text{if tie} \end{cases}$

3.3 Hyperparameters

$\gamma = 0.95, \alpha = 0.1, \epsilon \leftarrow \max(0.01, \epsilon \cdot 0.999)$

4 Core Module Details

4.1 game/constants.py · Game Constants

Centralized game parameters (single point of modification):

- **SUITS** = ['s', 'h'] · spades, hearts
- **RANKS** = ['7', '8', '9', 'T', 'J', 'Q', 'K', 'A'] · 8 ranks
- **BETTING_LEVELS** = [10, 20, 40, 80] · 4-level betting
- **MAX_RAISES** = 3 · per-round raise limit
- **Action codes**: FOLD=0, CALL=1, RAISE=2
- **Round codes**: PREFLOP=0, FLOP=1, TURN=2, RIVER=3

4.2 game/card.py · Cards & Deck

Built on treys library's `Card.new()` for integer card representation:

- **Deck class**: Initialize 16 cards · `reset()` shuffle · `deal(n)` draw
- **Utility functions**: `card_to_str`, `card_to_pretty`, `build_full_deck()`

4.3 game/evaluator.py · Hand Evaluation & Comparison

Function	Description	-----
-----	evaluate_hand(hole, community)	Returns (rank, class_str); lower rank = stronger
compare_hands(hole1, hole2, community)	Returns (1/0/-1, winner_hand_class)	
compute_equity(hole, community, sim=0)	Win rate; sim=0 = exact enum, else MC sample	
equity_to_bin(equity, bins=20)	Discretize [0,1] equity to bin index	

The 16-card deck keeps exact enumeration computationally feasible.

4.4 game/engine.py · Game Engine

Data structures:

- **PlayerState**: chips, hole cards, bets, fold status
- **Observation**: Agent-observable state (corresponds to MDP state s)

- **HandResult**: Settlement result of one hand

Two calling interfaces:

1 Step-based (RL training):

```
obs = engine.reset_hand()
obs, reward, done, info = engine.step(action)
```

2 Hand-based (evaluation/tournament):

```
result = engine.run_hand()
results = engine.run(1000)
```

Betting logic:

- Preflop: `betting_level` starts at 0 (BB=10 already posted), can raise from level 0
 - Post-flop: `betting_level = -1` means "no bet yet"; first Raise sets level 0, Call acts as Check
 - Each round resets `round_bet`, `acted_this_round`, `raises_this_round`
-

5 Agent Interface Design

5.1 BaseAgent Abstract Class

```
class BaseAgent(ABC):
    def act(self, obs: Observation) -> int          # Required: select
    action
    def reset(self) -> None                          # Optional: reset
    internal state
    def update(self, obs, action, reward, next_obs, done) #
    Optional: online learning
```

5.2 Observation Fields

Fields available to Agent in `act()`:

Field	Type	Description
<code>hole_cards</code>	<code>list[int]</code>	Own hole cards (treys integers)
<code>community_cards</code>	<code>list[int]</code>	Current community cards
<code>pot</code>	<code>int</code>	Total pot
<code>current_bet</code>	<code>int</code>	Current amount to call
<code>player_chips</code>	<code>int</code>	Own remaining chips
<code>opponent_chips</code>	<code>int</code>	Opponent remaining chips
<code>betting_level</code>	<code>int</code>	Betting level (0~3)
<code>current_round</code>	<code>int</code>	Current round (0~3)
<code>position</code>	<code>int</code>	Seat (0 or 1)
<code>legal_actions</code>	<code>list[int]</code>	Legal action list
<code>raises_this_round</code>	<code>int</code>	Raises made this round
<code>equity</code>	<code>float</code>	Current hand equity

5.3 Agent Summary

Agent	Core Method	State Encoding	Expert	External
Sampling MCCFR · approximate Nash eq.	<code>Q(s,a)</code> online TD update	<code>(H_code, P_code, B_level, Pos)</code>	SARSA	
Bayesian inference on opponent hand · MC Q-table	<code>(S, B, O)</code> , <code>O = argmax</code> posterior		Bayesian-MC	
BNN (MC Dropout) predicts opponent · MC Q-table	<code>(S, B, NN)</code> , <code>O_NN = BNN argmax</code>		NN-MC	

6 Expert Agent Implementation

6.1 CFR Algorithm Core

Counterfactual Regret Minimization (CFR) iteratively minimizes "counterfactual regret" to approximate Nash equilibrium:

- 1 **Counterfactual regret:** Extra utility gained at info set I if player always chose action a instead of current strategy
- 2 **Regret Matching:** Compute strategy from cumulative regrets; more-regretted actions become less likely
- 3 **Average strategy:** Weighted average of all iteration strategies converges to Nash equilibrium

6.2 External Sampling MCCFR

To reduce traversal cost, we use the **External Sampling** variant:

Decision Type	Processing Method	Current player action
Traverse all legal actions	Opponent action	Sample one via strategy
Sample community cards		Dealing (Chance)

This reduces per-iteration node count from exponential to linear. 30k iterations · 2 minutes.

6.3 Information Set Design

```
info_key = (player, hole_bucket, community_bucket,
            betting_round, betting_level, raises_this_round)
```

Component	Encoding	Description
player	0/1	Player ID
hole_bucket	0~9	Preflop equity discretized into 10 buckets
community_bucket	0/3/4/5	Community card count (encodes game stage)
betting_round	0~3	Preflop/Flop/Turn/River
betting_level	-1~3	Bet level (-1=no bet yet, 0~3=levels)
raises_this_round	0~3	Raises made this round

Key design decision: In Limit Hold'em, betting state is fully determined by (betting_level, raises_this_round) · no need for full action history. This reduces info set count from hundreds of thousands to ~200.

6.4 Preflop Equity Cache

120 hole card combinations ($C(16,2)$) are pre-computed at first training:

- MC sampling (per opponent hand × 8 board samples) instead of exact enumeration
- ~0.02s per combo, ~2s total for full cache
- O(1) info set lookup after caching

6.5 Benchmark Results

500-hand match statistics (30k CFR iterations):

Matchup	Win Rate	Notes
Expert vs Random	67.8% vs 28.6%	CFR significantly beats random
Expert vs Expert	45.6% vs 49.0%	
Near 50/50 (slight position bias)	Random vs Random	~50/50
Symmetric control		

7 Design Rationale for Simplified Rules

The 52·16 card compression + discrete betting limits simultaneously solve two computational bottlenecks:

- 1 **Hand evaluation enumeration explosion:** treys equity computation needs to enumerate possible board completions. With 52 cards the combinations are too large (e.g., $C(45,2)$). 16 cards makes exact enumeration feasible.
- 2 **Bayesian posterior enumeration infeasible:** Bayesian-MC Agent must enumerate all opponent hand combinations for posterior computation. 16 cards enables online marginalization.

This setup is equivalent to a well-studied *Limit Texas Hold'em* variant, preserving the core structure of imperfect-information games.

8 How to Run & Test

8.1 Environment Setup

```
# Install dependencies (recommended: use conda environment)
pip install -r requirements.txt
```

The only dependency is `treys` (poker hand evaluation library).

8.2 Available Agent Types

Agent Key	Class	Description	----- -----
		random	RandomAgent Uniform random over legal actions
expert	ExpertAgent	CFR-trained Nash equilibrium strategy	sarsa SARSAAgent SARSA
		online Q-learning (placeholder)	bayesian_mc BayesianMCAgent Bayesian + MC Q-table (placeholder)
nn_mc	NN_MCAgent	BNN + MC Q-table (placeholder)	

8.3 Running from Command Line

```
# Mode 1: Interactive · watch a detailed step-by-step game
python main.py --mode interactive --num_hands 5
# Mode 2: Evaluate · batch compare two agents (statistics output)
python main.py --mode evaluate --agent0 expert --agent1 random
--num_hands 1000
# Mode 3: Step · debug RL step interface (observation details)
python main.py --mode step
```

Example output (evaluate mode):

```
=====
Evaluation: expert_p0 vs random_p1
Total hands: 1000
=====
Player 0 (expert_p0) wins: 678 (67.8%)
Player 1 (random_p1) wins: 286 (28.6%)
Ties: 36 (3.6%)
Avg reward P0: 12.35
Avg reward P1: -12.35
```

8.4 Running from Python Code

```

from game.engine import GameEngine
from agents.expert_agent import ExpertAgent
from agents.random_agent import RandomAgent
# Create agents
expert = ExpertAgent(name="CFR_Expert")
random_ag = RandomAgent(name="Baseline")
# Run 500 hands
engine = GameEngine(expert, random_ag)
results = engine.run(num_hands=500)
# Analyze results
wins = sum(1 for r in results if r.winner == 0)
print(f"Expert win rate: {wins / len(results) * 100:.1f}%")

```

8.5 Step-by-Step RL Interface (for Training)

```

from game.engine import GameEngine
from agents.random_agent import RandomAgent
agent0 = RandomAgent(name="Learner")
agent1 = RandomAgent(name="Opponent")
engine = GameEngine(agent0, agent1)
# Reset a new hand
obs = engine.reset_hand()
print(f"Hole cards: {obs.hole_cards_pretty}")
print(f"Legal actions: {obs.legal_actions}")
print(f"Equity: {obs.equity:.4f}")
# Take actions step by step
done = False
while not done:
    action = agent0.act(obs) if engine.current_player == 0 else
    agent1.act(obs)
    obs, reward, done, info = engine.step(action)
# Get result
result = info["result"]
print(f"Winner: Player {result.winner}, Pot: {result.pot}")

```

8.6 Training Expert Agent from Scratch

```

from agents.expert_agent import ExpertAgent
# Train CFR for 50000 iterations (takes ~3 minutes)
agent = ExpertAgent(train_iterations=50000)
# Policy auto-saved to cfr_policy.pkl
# Next time, auto-loads from file (instant)
agent = ExpertAgent()

```

8.7 Quick Test Commands

```

# Test random vs random (sanity check . should be ~50/50)
python main.py --mode evaluate --agent0 random --agent1 random
--num_hands 500
# Test expert vs random (expert should win ~65-70%)
python main.py --mode evaluate --agent0 expert --agent1 random
--num_hands 500

```

```
# Watch a single hand interactively  
python main.py --mode interactive --num_hands 1
```